

AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Api Documentation

Table of Contents

- REST API Documentation
 - Endpoint Reference
 - Python Analytics API Reference
 - Validation Evidence
 - Role Test Seed
 - JSON Examples

REST API Documentation

Node API base URI: `/api/v1`

Python Analytics API base URI: `/api/analytics/v1`

Common headers:

- `Content-Type: application/json`
- Authenticated endpoints require cookie `alc_session` or `Authorization: Bearer <session-token>`.

Response envelope:

```
{ "success": true, "message": "OK", "data": {} }
```

```
{ "success": false, "message": "Validation failed", "details": {} }
```

Common status codes: `200`, `201`, `400`, `401`, `403`, `404`, `409`, `422`, `500`.

Endpoint Reference

Method	URI	Description
GET	<code>/auth/config</code>	Public authentication configuration for the React login page.
POST	<code>/auth/login</code>	Postman/basic authentication with configuration email and password or seeded user password hash.
POST	<code>/auth/google</code>	Login/register with Google token.
GET	<code>/auth/me</code>	Current session user.
POST	<code>/auth/logout</code>	Revoke session and clear cookies.
POST	<code>/enrollment-requests</code>	Public enrollment request.
GET	<code>/enrollment-requests</code>	List public requests.
GET	<code>/users</code>	List users.
PATCH	<code>/users/{id}/role</code>	Assign internal role.

Method	URI	Description
GET	<code>/users/{id}/branch-access</code>	List branch access assigned to user.
PATCH	<code>/users/{id}/branch-access</code>	Replace branch access assigned to a user.
GET	<code>/roles</code>	List roles.
GET	<code>/permissions</code>	List permissions.
GET	<code>/branches</code>	List branches.
POST	<code>/branches</code>	Create branch.
PATCH	<code>/branches/{id}</code>	Update branch.
GET/POST/PATCH	<code>/students</code>	List/create/update students.
GET/POST/PATCH	<code>/teachers</code>	List/create/update teachers.
GET/POST/PATCH	<code>/dance-categories</code>	Manage dance categories.
GET/POST/PATCH	<code>/dance-styles</code>	Manage dance styles.

Method	URI	Description
GET/POST/PATCH	<code>/class-groups</code>	Manage class groups.
GET/POST/PATCH	<code>/class-sessions</code>	Manage scheduled sessions.
POST	<code>/student-attendance</code>	Record student attendance.
POST	<code>/teacher-attendance/check-in</code>	Teacher check in.
PATCH	<code>/teacher-attendance/{id}/check-out</code>	Teacher check out.
GET	<code>/absence-justifications</code>	List absence justifications.
POST	<code>/absence-justifications</code>	Submit absence justification.
PATCH	<code>/absence-justifications/{id}/review</code>	Approve/reject justification.
GET	<code>/reports/branches/summary</code>	Branch summary report.

Method	URI	Description
GET	<code>/reports/scholarships/{studentId}/candidate?from=&to=</code>	Scholarship candidate ch
POST	<code>/scholarship-evaluations</code>	Register scholarship evaluation.
GET	<code>/scholarship-evaluations</code>	List scholars evaluations.
GET	<code>/reports/level-promotions/{studentId}/candidate?from=&to=</code>	Level promo candidate ch
POST	<code>/level-promotion-evaluations</code>	Register promotion evaluation.
GET	<code>/level-promotion-evaluations</code>	List promoti evaluations.
GET	<code>/reports/teachers/{teacherId}/payment</code>	Teacher paym calculation.
GET	<code>/audit-logs</code>	List audit log

Python Analytics API Reference

The Python Analytics API is implemented with FastAPI under `06Code/python-analytics-api`. It reuses the JWT session token issued by the Node Auth API and enforces the same resource-scope policy for students, teachers and assigned branches.

Method	URI	Description
GET	<code>/api/analytics/v1/health</code>	Python API health check.

Method	URI	Description
GET	<code>/api/analytics/v1/students/{student_id}/attendance-risk</code>	Student attendance risk analytics.
GET	<code>/api/analytics/v1/students/{student_id}/scholarship-readiness</code>	Scholarship readiness analytics.
GET	<code>/api/analytics/v1/branches/{branch_id}/performance-summary</code>	Branch performance analytics.
GET	<code>/api/analytics/v1/teachers/{teacher_id}/workload-summary</code>	Teacher workload analytics.

Validation Evidence

- Automated API validation command: `cd 06Code && npm run test:api:validation`
- Latest generated report: `03Documentation/api-validation-report.md`
- Latest JSON result evidence: `070ther/api-validation-results.json`
- Manual Postman verification assets: `postman/American-Latin-Class-API.postman_collection.json`, `postman/American-Latin-Class-Analytics-API.postman_collection.json` and `postman/American-Latin-Class.postman_environment.json`

The automated validation covers auth config, Postman email/password login, malformed Google token rejection, Google session creation with mock test tokens, JWT Bearer access, session revocation, revoked token rejection, anonymous/private redirects, RBAC failures, public enrollment validation, CRUD flows, attendance duplication rules, teacher check-in/check-out, absence review, reports, scholarship evaluations, promotion evaluations and audit log visibility.

Role Test Seed

Run this after applying the database schema when you need manual role/permission testing:

```
cd 06Code
npm run db:seed:role-test
```

The seed creates/update temporary login users for Admin, GeneralDirector, BranchDirector, Teacher and Student. Default local passwords are `adminALC2026*`, `generaldirectorALC2026*`, `branchdirectorALC2026*`, `teacherALC2026*` and `studentALC2026*`; override them with `SEED_*_PASSWORD` environment variables before running the seed. The BranchDirector user is assigned only to `ALC Santo Domingo Norte`, leaving `ALC Santo Domingo Central` available for negative permission tests.

JSON Examples

Postman password login:

```
{
  "email": "verification-admin-real-20260624154645@alc.test",
  "password": "{{login_password}}"
}
```

Google login:

```
{
  "idToken": "eyJhbGciOi..."
}
```

Login response:

```
{
  "success": true,
  "message": "Login successful",
  "data": {
    "sessionToken": "eyJhbGciOi...",
    "tokenType": "Bearer",
    "expiresInMinutes": 120,
    "user": {
      "id": "d8c025f6-bcd0-4f25-a6b1-1486338678e7",
      "email": "student@example.com",
      "name": "Student Example",
      "role": "Student"
    }
  }
}
```

Postman authenticated request:

```
GET /api/v1/auth/me
Authorization: Bearer {{session_token}}
```

Logout invalidates the backend session:

```
POST /api/v1/auth/logout
Authorization: Bearer {{session_token}}
```

After logout, the same token must return **401**:

```
GET /api/v1/auth/me
Authorization: Bearer {{session_token}}
```

Python analytics authenticated request:

```
GET /api/analytics/v1/students/{{student_id}}/attendance-risk
Authorization: Bearer {{session_token}}
```

Python analytics unauthorized response:

```
{
  "success": false,
  "message": "Authentication required",
  "data": null
}
```

Student attendance:

```
{
  "studentId": "85f4bbe9-5d5f-4126-89b6-ddd9de432885",
  "classSessionId": "76f37581-dbbc-4201-bb13-67fbc86f6d60",
  "status": "present",
  "notes": "On time"
}
```

Scholarship evaluation:

```
{
  "studentId": "85f4bbe9-5d5f-4126-89b6-ddd9de432885",
  "percentage": 50,
  "theoryScore": 82,
  "practiceScore": 91,
  "approved": true
}
```


Error response:

```
{  
  "success": false,  
  "message": "Insufficient permissions",  
  "details": null  
}
```